Ders 15 Genişletilmiş Notu: Öncelikli Kuyruklar ve Heap Alıştırmaları

Ders 15 Genişletilmiş Notu: Öncelikli Kuyruklar ve Heap Alıştırmaları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Öncelikli Kuyruk ve Gerçekleme Seçenekleri
- 2. Öncelikli Kuyruk İşlemleri ve Karmaşıklıkları
- 3. Öncelikli Kuyruğun Kullanım Alanları
- 4. Binary Heap ile Ekleme ve Çıkarma: Banka Örneği
- 5. Kod Üzerinden Ekleme, Heapify ve Kök Silme
- 6. Alıştırma 1: Bir İkili Ağaç Heap mi?
- 7. Alıştırma 2: Sırasız Dizide k'ıncı En Küçük Eleman
- 8. Alıştırma 3: Min Heap'i Max Heap'e Dönüştürme
- Örnek Sorular
- Dersin Toparlaması

1. Öncelikli Kuyruk ve Gerçekleme Seçenekleri

Hoca derse, geçen hafta heap veri yapısını ve işlemlerini konuştuklarını hatırlatarak başladı. Bu haftanın konusu öncelikli kuyruktur (priority queue). Öncelikli kuyruk, kuyruğun özel bir hâlidir: elemanlar geliş sırasına göre değil, önceliklerine göre hizmet alır. Genellikle heap veri yapısıyla gerçeklenir (implement edilir). Ama tek seçenek heap değildir. Derste bu alternatiflerin neler olduğu, hangisinin neden tercih edilip edilmediği konuşuldu.

Hoca bu bölümü öğrencilere soru sorarak yürüttü ve bunun bir amacı olduğunu söyledi.

> **Hocanın vurgusu:** Bu sorulara cevap vermeye çalışmak sınav için antrenman yapmak demektir. Bu tür sorularla sınavda da karşılaşma ihtimali vardır.

1.1 Heap Dışında Hangi Yapılar Kullanılabilir?

Hocanın sorusu şuydu: Öncelikli kuyruk için heap kullanabileceğimizi söyledik, başka ne kullanabiliriz?

- **Yığın (stack):** Bir öğrenci yığın önerdi. Hoca yığını tercih etmediklerini söyledi, çünkü yığınla istenen verimlilikte bir öncelikli kuyruk kurulamaz. Hoca, daha önce bir kuyruğun iki yığınla gerçeklenebileceğini konuştuklarını da hatırlattı. Ama burada aranan, temel veri yapıları (elementary data structures) dediğimiz daha basit yapılardır.
- **Bağlı liste (linked list):** Bir öğrencinin önerisi. Hoca doğru olduğunu söyledi.
- **Dizi (array):** Bir başka öğrencinin önerisi. Bu da doğrudur.
- **İkili arama ağacı (binary search tree):** Hoca bir yapı daha olduğunu, birkaç hafta önce görüldüğünü söyledi. İpucu olarak, öncelikli kuyruk denince genelde en yüksek değerin aklımıza geldiğini, en yüksek değere hangi yapının bizi ulaştırabileceğini sordu. Bir öğrenci "ağaç" dedi. Hoca "ağacın özel bir hâli" diyerek ikili arama ağacına (BST) vardı.

İkili arama ağacı söz konusu olunca karşımıza daha çok AVL ve benzeri ağaçlar çıkar, çünkü ağacın dengeli olması önemlidir.

> **Hocanın vurgusu:** Dengesiz bir ağaçta öncelikli kuyruk oluşturmanın pek bir faydası yoktur. Dengeli (AVL gibi) bir ağaç gerekir.

2. Öncelikli Kuyruk İşlemleri ve Karmaşıklıkları

Yapıları verimlilik açısından karşılaştırmak için önce öncelikli kuyrukta hangi işlerin yapıldığını belirlemek gerekir. Hoca üç temel işlemi saydı:

- **`insert`:** Öncelikli kuyruğa eleman eklemek. Başka bir deyişle `enqueue`.
- **`remove`:** Kuyruktan işlem görecek, yani en öncelikli elemanı almak. Bunun bir başka adı `delete`'tir.
- **`peek`:** En öncelikli sırada bekleyen elemanın değerine ulaşmak. Eleman kuyruktan çıkarılmaz.

2.1 Dizi ile Gerçekleme

Hoca, öncelikli kuyruğu (sırasız) bir diziyle gerçeklersek karmaşıklıkların ne olacağını sordu.

- **`insert`:** Ekleme dizinin sonuna yapılır, $O(1)$.
- **`remove`:** En yüksek öncelikli elemanı çıkarmak için önce onu bütün dizide bulmak gerekir, $O(n)$.
- **`peek`:** En öncelikli elemanın değerine ulaşmak için yine bütün dizi taranır, $O(n)$.

2.2 Bağlı Liste ile Gerçekleme

Bağlı listede tasarım tercihi önemlidir. Hoca, elemanı önceliğine göre listenin en önüne (ya da en sonuna) koymayı hedeflediğimiz sıralı bir bağlı liste düşündü.

- **`insert`:** Liste her eleman geldikçe sıralı tutulduğu için, yeni elemanın yeri bulunana kadar elemanlar tek tek kontrol edilir. Gelen eleman listedeki en büyük elemandan da büyükse onun önüne konur. En kötü durumda n karşılaştırma gerekir, $O(n)$.
- **`remove`:** Elemanlar sırayla çıkarıldığı için en öncelikli eleman listenin başındadır, $O(1)$.
- **`peek`:** Aynı sebeple $O(1)$.

2.3 Dengeli İkili Arama Ağacı ile Gerçekleme

Hoca AVL gibi dengeli bir ağaç için karmaşıklıkları sordu. Öğrenciler `insert` için $O(\log n)$, `remove` için de $O(\log n)$ dedi. Hoca ikisini de onayladı. Sohbetten (chat) de aynı cevap gelmişti.

2.4 Karşılaştırma Tablosu

Hoca bu konuşmadan sonra slaydı kaydırdı. Cevaplar slaytta yazdığı için özellikle bu slaytla başlamak istemediğini söyledi. Slayttaki tablo şöyle özetlenebilir:

Gerçekleme	`insert`	`remove` (`delete`)	`peek`
Sırasız dizi	$O(1)$	$O(n)$	$O(n)$
Sıralı bağlı liste	$O(n)$	$O(1)$	$O(1)$
Binary heap	$O(\log n)$	$O(\log n)$	$O(1)$
İkili arama ağacı (dengeli)	$O(\log n)$	$O(\log n)$	slaytta $O(1)$

Hoca slayttaki "multidimensional array" ifadesinin yerine yalnız "array" demenin daha doğru olacağını söyleyip düzeltti. Binary heap'te `peek`'in $O(1)$ olmasının sebebi, en öncelikli elemanın her zaman en üst noktada, yani kökte yer almasıdır.

2.5 İkili Arama Ağacında peek: $O(1)$ mi, $O(\log n)$ mi?

Tabloda dikkat çeken bir nokta vardı. Az önce BST'de peek için $O(\log n)$ demişlerdi, slaytta ise $O(1)$ yazıyordu. Hoca hangisinin doğru olduğunu sordu ve bir BST çizerek hatırlattı: büyük değerler bir tarafa, küçükler öbür tarafa yerleşir. Çizimde 15, 25, 9 ve 6 gibi değerler vardı.

Bir öğrenci sohbette $O(1)$ cevabını verdi. Hoca sebebini sordu ama cevap gelmedi. Hoca doğru cevabın $O(1)$ olduğunu söyleyip tabloyu işaretledi ve konuya dersin ilerleyen dakikalarında döneceğini söyledi.

> **Hocanın vurgusu:** Bu kısım önemli. Bu noktalara gerçekten kafa yormak gerekiyor.

Aradan sonra hoca açıklamasını yaptı. BST'de en sağda yer alan düğüm her zaman en yüksek değeri gösterir. Örneğin 25 çıkarılırsa 20, 20 çıkarılırsa 10 en yüksek değer olarak karşımıza çıkar. Hocaya göre en sağdaki elemana doğrudan erişilebilir, bu yüzden en yüksek değere tek adımda ulaşılır ve peek $O(1)$'dir.

delete'in farkı şudur: Bir elemanı sildiğinizde ağaçta yeniden bir düzenleme yapmanız gerekebilir. Bu düzenlemenin maliyeti en kötü durumda $O(\log n)$'dir. Yani peek doğrudan en yüksek değere erişmektir. delete'te de değere erişirsiniz, ama eriştikten sonra onu kuyruktan çıkarmak için harcanan emek $O(\log n)$ karmaşıklığındadır.

> **Ders sonrası not:** Hoca, en sağdaki düğüme "doğrudan" erişildiği için peek'in $O(1)$ olduğunu söyledi. En sağdaki düğüme ulaşmak için kökten başlayıp sağ çocuklar boyunca inmek gerekir. Dengeli bir ağaçta bu yol $O(\log n)$ uzunluğundadır. peek'in $O(1)$ olması ancak en büyük düğümü gösteren ayrı bir işaretçi tutulur ve her ekleme ve silmede güncellenirse mümkündür. Slayttaki $O(1)$ değeri bu ek işaretçi varsayımıyla doğrudur.

2.6 Neden Binary Heap?

Hoca, gördüklerimize bakınca öncelikli kuyruğu genellikle binary heap üzerinden modellemenin karmaşıklık açısından önemli avantajlar sağladığını söyledi. Ekleme ve çıkarma işlemleri $O(\log n)$ seviyesine iner. Heap veri yapısının en temel amacı da en yüksek öncelikli (ya da en düşük değerli) elemana $O(1)$ karmaşıklığında ulaşmaktır. Binary heap bunu mümkün kılar.

3. Öncelikli Kuyruğun Kullanım Alanları

Hoca geçen hafta da biraz bahsettikleri kullanım alanlarını saydı:

- **İşletim sisteminde süreç yönetimi (process management) ve kesme işleme (interrupt handling):** Süreçlerin öncelikleri varsa, ya da o sırada gerçekleşen bir işlemin diğerlerinden önce yapılması gerekiyorsa (buna kesme, interrupt deniyor) öncelikli kuyruktan faydalanılır.
- **Dijkstra algoritması:** En kısa yol bulma algoritmalarından biridir ve öncelikli kuyruk kullanır.
- **Huffman kodlaması (Huffman coding):** Öğrencilerin daha önce ayrık matematikte görmüş olabileceği bu yöntemde de öncelikli kuyruktan faydalanılır.
- **Banka:** Bazı müşterilerin önceliği vardır. Bu, yaştan ya da bankanın özel müşterisi olmaktan kaynaklanabilir. Sistem sıradaki kişiyi seçerken öncelik sırasını gözetir.
- **Uçak:** Business ve economy sınıfıyla ilgili işlerde de çeşitli öncelikler kullanılarak kuyruklar oluşturulur ve müşterinin hizmet alması sağlanır.

4. Binary Heap ile Ekleme ve Çıkarma: Banka Örneği

Hoca, geçen hafta konuşulan ekleme işlemini hatırlatma amacıyla bir max heap üzerinde yeniden gösterdi. Heap'in dizi hâli şöyleydi:

$$9,\ 3,\ 5,\ 1,\ 4,\ 2$$

Kökte 9 olduğu için sıradaki hizmet alacak kişi 9 numaralı müşteridir.

4.1 Ekleme: 11 Numaralı Müşteri Geliyor

Bankaya yeni bir müşteri gelir, otomatikten bir bilet alır ve önceliği 11'dir. Ekleme şöyle ilerler (0 tabanlı dizi):

Adım	Dizi	Açıklama
Başlangıç	9, 3, 5, 1, 4, 2	Kökte 9
11 eklenir	9, 3, 5, 1, 4, 2, 11	11 son boş göze (indeks 6) konur
1. yer değiştirme	9, 3, 11, 1, 4, 2, 5	Ebeveyn 5 < 11, 11 yukarı çıkar, 5 aşağı iner
2. yer değiştirme	11, 3, 9, 1, 4, 2, 5	Ebeveyn 9 < 11, 11 köke çıkar

Heapify işlemi tamamlandığında 9 önceliğini kaybeder. 11 daha öncelikli olduğu için en yukarıdadır. Artık ilk çağrılacak ve ilk hizmet alacak kişi 11 numaralı müşteridir. Hoca, öncelikli kuyruğun tam olarak bu olduğunu söyledi: gelen kişi yeniden konumlandırılır, diğerlerinin yerleri değiştirilir ve böylece kimin hizmet alacağını belirleyen sıra oluşur.

4.2 Çıkarma: 11 Numaralı Müşteri Hizmet Alıyor

Şimdi 11 numaralı müşterinin hizmet alıp kuyruktan ayrıldığını düşünelim. Kuyruktan çıkarma (remove) şöyle yapılır:

Adım	Dizi	Açıklama
Başlangıç	11, 3, 9, 1, 4, 2, 5	Kökte 11
Kök ile son eleman yer değiştirir	5, 3, 9, 1, 4, 2 | 11	11 sona kayar, 5 yukarı gelir. Eleman sayısı $n-1$ olur, 11 artık kuyrukta değildir
Kökten heapify	9, 3, 5, 1, 4, 2	5'in çocukları 3 ve 9; 9 daha büyük, 9 yukarı çıkar
Devam	9, 3, 5, 1, 4, 2	5'in yeni konumunda tek çocuğu 2 var; 2 daha küçük olduğu için 5 yerinde kalır

Böylece öncelikli kuyruğun yeni hâli oluşur ve sıradaki müşteri yine 9 numaralı müşteridir. Biraz önce o sırada bekliyordu, ama 11 gelince geriye düşmüştü. Şimdi 11 hizmet aldığı için sıra tekrar ona geldi.

Hocanın ifadesiyle bu işlemin iki adımı vardır: önce kuyruktan çıkan eleman son elemanla değiştirilir, sonra heapify işlemi yapılır.

> **Ders sonrası not:** Dizi, altyazıda okunduğu sırayla (9, 3, 5, 1, 4, 2) alınırsa geçerli bir max heap değildir: 1 numaralı gözdeki 3'ün sağ çocuğu (4 numaralı göz) 4'tür ve $4 > 3$'tür. Ekleme ve çıkarma izinde 11 ve 5 yalnız 2 numaralı göz tarafında hareket ettiği için bu ihlal izin sonucunu değiştirmez. Tahtadaki ağaçta 3 ile 4'ün yerleri farklı çizilmiş olabilir.

5. Kod Üzerinden Ekleme, Heapify ve Kök Silme

Hoca ardından slayttaki kod parçalarının üzerinden geçti.

5.1 Slayttaki `insert`

Fonksiyona kuyruğu temsil eden bir dizi ve eklenecek yeni müşterinin numarası gönderilir.

- Bekleyen hiç eleman yoksa (`size == 0`) yeni müşterinin numarası ilk göze yazılır ve `size` bir artırılır.

- Sırada müşteri varsa (`else` kolu) yeni müşteri dizinin sonuna konur, `size` bir artırılır ve `size / 2 - 1`'den başlayarak köke kadar her düğüm için heapify çağrılır.

Hoca burada öğrencilere, biraz önce tahtada yaptıkları ekleme ile bu kod arasında küçük bir fark olduğunu söyleyip bu farkın ne olduğunu sordu. Cevap gelmeyince ilk anda kodu savunur gibi anlattı: ekleme yapılınca heapify'ın en sondaki iç düğümden (internal node) başlatılması, köke kadar her düğüm gezilerek elemanların yerlerine yerleştirilmesi gerektiğini söyledi.

5.2 Heapify

Heapify'ı geçen hafta görmüşlerdi. Hoca adımları şöyle anlattı:

- Tek bir eleman varsa (`size == 1`) yapılacak bir şey yoktur.
- Değilse, `i` noktasındaki eleman en büyük (`largest`) kabul edilir.
- Sol çocuk (`2i + 1`) `largest`'tan büyükse `largest` sol çocuk olur.
- Sağ çocuk (`2i + 2`), o anki `largest`'tan da büyükse, yani hem ebeveynden hem sol çocuktan büyükse, `largest` sağ çocuk olur.
- Çocuklardan biri ebeveynden büyükse yer değiştirme (swap) yapılır ve heapify, aşağı inen eleman için kendini tekrar çağırır.

5.3 Kök Silme (`deleteRoot`)

Silme, `remove` işlemidir. Çıkarılması istenen eleman öncelikli kuyruktaki sıradaki elemandır, yani kökteki 0 numaralı elemandır. Bu eleman son elemanla yer değiştirir, `size` bir azaltılır ve heapify işlemiyle yapı yeniden düzenlenir.

5.4 Ekleme İçin Bütün İç Düğümleri Gezmek Gerekmez

Aradan sonra hoca, insert kodunda bilerek yanlış anlattığı noktaya döndü. 11 eklendiğinde bir heapify yapıldığını, ama insert anlatılırken bütün iç düğümlerin heapify edilmesi gerektiğinin söylendiğini hatırlattı ve bunun doğru olmadığını açıkladı.

> **Hocanın vurgusu:** Önemli olan nokta buydu. Birinin dikkatini çekmesini bekledim, ama kimseden cevap gelmedi.

Ağaç zaten düzenli, yani zaten bir heap olduğu için 11 eklendiğinde tahtada yapıldığı gibi aşağıdan yukarı (bottom-up) bir heapify yeterlidir. 11 olması gereken yere yerleşir. Diğer iç düğümleri tek tek gezmeye gerek yoktur. `for` döngüsüyle bütün iç düğümleri gezme işi, sıfırdan yeni bir heap ağacı kurulurken ya da heap sort yapılırken kullanılır. Orada da dizinin ortasındaki (yarısındaki) elemandan başlanır. Insert işleminde döngü kullanmadan yalnız ilgili düğümün heapify edilmesi yeterlidir.

5.5 Kodun Tamamı

Aşağıdaki program, slayttaki ilk insert sürümünü (`insertSlide`), yeterli olan aşağıdan yukarı sürümü (`insert`), heapify'ı ve kök silmeyi bir arada verir. Kod, anlatımdan kurulmuştur ve `gcc -Wall` ile uyarısız derlenip çalıştırılmıştır.

```c
#include <stdio.h>

void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }

/* Yukaridan asagi max heapify (0 tabanli dizi) */
void heapify(int heap[], int size, int i) {
    if (size == 1) return;
    int largest = i;
    int l = 2 * i + 1;
    int r = 2 * i + 2;
    if (l < size && heap[l] > heap[largest]) largest = l;
    if (r < size && heap[r] > heap[largest]) largest = r;
    if (largest != i) {
        swap(&heap[i], &heap[largest]);
```

```
        heapify(heap, size, largest);
    }
}

/* Slayttaki ilk surum: eklemeden sonra butun ic dugumleri heapify eder */
void insertSlide(int heap[], int *size, int value) {
    if (*size == 0) {
        heap[0] = value;
        (*size)++;
    } else {
        heap[*size] = value;
        (*size)++;
        for (int i = *size / 2 - 1; i >= 0; i--)
            heapify(heap, *size, i);
    }
}

/* Yeterli surum: yalniz yeni eleman asagidan yukari tasinir */
void insert(int heap[], int *size, int value) {
    int i = (*size)++;
    heap[i] = value;
    while (i > 0 && heap[(i - 1) / 2] < heap[i]) {
        swap(&heap[i], &heap[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}

/* Kok silme: kok son elemanla yer degistirir, size azalir, kokten heapify */
int deleteRoot(int heap[], int *size) {
    int root = heap[0];
    swap(&heap[0], &heap[*size - 1]);
    (*size)--;
    heapify(heap, *size, 0);
    return root;
}

void print(const char *s, int heap[], int size) {
    printf("%-14s", s);
    for (int i = 0; i < size; i++) printf("%d ", heap[i]);
    printf("\n");
}

int main(void) {
    int a[10] = {9, 3, 5, 1, 4, 2}, na = 6;
    int b[10] = {9, 3, 5, 1, 4, 2}, nb = 6;
    insert(a, &na, 11);
    insertSlide(b, &nb, 11);
    print("insert:", a, na);
    print("insertSlide:", b, nb);
    int r = deleteRoot(a, &na);
    printf("silinen: %d\n", r);
    print("deleteRoot:", a, na);
    return 0;
}
```

Çıktı:

```
insert:       11 3 9 1 4 2 5
insertSlide:  11 4 9 1 3 2 5
silinen: 11
deleteRoot:   9 3 5 1 4 2
```

`insert` çıktısı 4.1'deki izle, `deleteRoot` çıktısı 4.2'deki izle aynıdır. `insertSlide` çıktısının farklı olmasının sebebi, başlangıç dizisindeki 3 ile 4 ihlalidir (bkz. 4.2'deki ders sonrası not): bütün iç düğümleri gezen döngü bu ihlali de düzeltir. Başlangıç dizisi gerçekten bir heap olsaydı iki sürüm aynı heap özelliğini sağlardı, ama döngülü sürüm her eklemede $O(n)$ iş yapardı.

> **Ek bilgi (derste anlatılmadı):** 0 tabanlı dizide `i` indeksindeki düğümün çocukları `2i + 1` ve `2i + 2`, ebeveyni `(i - 1) / 2` (tam sayı bölmesi) indeksindedir. Aşağıdan yukarı yerleştirmede yeni eleman en fazla ağacın yüksekliği kadar, yani $O(\log n)$ adım yukarı çıkar. Döngülü sürüm ise yaklaşık $n/2$ iç düğümün her birinde heapify çağırdığı için eklemeyi $O(n)$'e çıkarır.

Hoca bununla öncelikli kuyrukla ilgili konuşulacakların bittiğini söyledi ve geçen haftanın alıştırma sorularına geçti.

6. Alıştırma 1: Bir İkili Ağaç Heap mi?

Geçen hafta, verilen bir ikili ağacın (binary tree) heap özelliği taşıyıp taşımadığını konuşalım denmişti. Hoca bunu ekranda bir C kodu üzerinden anlattı. Kod iki temel kontrol yapar:

- Ağacın complete binary tree olup olmadığı, yani düğümlerin soldan sağa boşluksuz dolup dolmadığı ("sola yaslı" olup olmadığı).
- Elemanların gerçekten heap kuralına göre dizilip dizilmediği.

6.1 Veri Yapısı ve Düğüm Sayma

`struct Node` yapısında düğümün taşıdığı değer ile `left` ve `right` adlı iki çocuk işaretçisi vardır. Yeni bir düğüm `malloc` ile yaratılır, değeri atanır ve çocuklarına `NULL` atanır.

`countNodes` fonksiyonu ağaçta kaç düğüm olduğunu sayar. `NULL` görene kadar hem sol hem sağ çocuk üzerinden özyinelemeli (recursive) olarak ilerler ve `NULL` olmayan her düğüm için bir ekler.

6.2 `isHeap`: İki Kontrolün Birleşimi

Ana fonksiyon `isHeap`'tir. Önce düğüm sayısını hesaplar, sonra `isCompleteUtil` ve `isHeapUtil` fonksiyonlarını çağırır. İkisi `&&` ile bağlandığı için `isCompleteUtil` `true` dönmezse `isHeapUtil` hiç çağrılmaz.

Fonksiyonların `bool` döndürmesinin sebebi, `bool` tipinin tanımlı olduğu özel bir başlık kütüphanesinin (`<stdbool.h>`) kullanılmasıdır.

6.3 `isCompleteUtil`

- Önce ağacın boş olup olmadığına bakılır. Boş ağaç complete özelliği taşır. Hocaya göre bu durum heap olup olmama açısından çok kritik değildir; yalnız boş bir ağaçtan söz ediyoruz.
- Ağaç boş değilse düğümün indeks numarasına bakılır. İndeks, düğüm sayısından büyük ya da eşitse `false` döner.
- Fonksiyon kendini sol çocuk için `2 * index + 1`, sağ çocuk için `2 * index + 2` indeksiyle çağırır. Yani her düğüme, dizideki gibi bir indeks verilir ve özyinelemeyle sola ve sağa dallanılır. Çocuk gördükçe bu sayılar büyür.

Mantığı şudur: Complete binary tree'de düğümler en yukarıdan başlayarak 1, 2, 3, 4, 5, 6, 7, 8 diye sırayla yerleştirilir. Arada bir tane bile boşluk olsa son düğüm farklı bir yere yerleşir. O zaman hesaplanan düğüm sayısından daha büyük bir indekse ulaşılır ve ağacın complete olmadığı anlaşılır.

> **Ders sonrası not:** Hoca, indeks düğüm sayısını aşınca "bu ağaç dengeli bir ağaç değil" dedi. Burada kontrol edilen özellik denge değil, ağacın complete olmasıdır. Dengeli olduğu hâlde complete olmayan ağaçlar vardır.

6.4 `isHeapUtil`

Bu fonksiyon max heap için yazılmıştır:

- Düğümün sol ve sağ çocuğu `NULL` ise tek bir düğüm, yani yaprak vardır. Bu bir heap'tir, `true` döner.
- Yalnız sağ çocuk `NULL` ise, ilk koşul sağlanmadığı için sol çocuk var demektir. Bu durumda ebeveynin değerinin sol çocuğun değerinden büyük (ya da eşit) olup olmadığına bakılır. Complete ağaçta yalnız sağ çocuğu olan bir düğüm olamayacağı için bu iki durum yeterlidir.
- İki çocuk da varsa (`else` kolu), ebeveynin değeri iki çocuğun değerinden de büyükse fonksiyon sol ve sağ alt ağaçlar için kendini çağırır. Böylece ağacın en alt kademesine kadar inilir.

Her kademedeki ebeveynin değeri çocuklarının değerinden büyükse hiçbir zaman `false` dönmez ve sonuç `true` olur. Min heap için karşılaştırmaların tersine çevrilmesi gerekir.

Hocanın özeti: Bir ikili ağacın heap olup olmadığını bulmak için iki kontrol vardır. Önce complete olup olmadığına bakılır; düğüm sayısından büyük bir indekse ulaşıldıysa ağaç complete değildir ve heap kontrolüne gerek kalmaz. Complete ise her ebeveynin çocuklarından büyük olup olmadığı tek tek kontrol edilir.

6.5 Kodun Tamamı ve Derste Yapılan Denemeler

Hoca kodu çalıştırdı. Seviye sırasıyla 10, 9, 8, 7, 6, 5, 4, 3, 2, 1 değerlerini taşıyan ağaç bir heap'ti. Ardından kökün değeri değiştirildi ve program ağacın heap olmadığını yazdı. Bir öğrenci kökün 10 yerine 0 kaldığını hatırlattı. Hoca kökü düzeltip bu kez aradaki bir düğümü çıkardı. Ağaç complete özelliğini kaybettiği için program yine heap olmadığını yazdı.

Aşağıdaki program anlatıma göre yazılmıştır ve `gcc -Wall` ile uyarısız derlenip çalıştırılmıştır.

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

struct Node {
    int key;
    struct Node *left, *right;
};

struct Node *newNode(int k) {
    struct Node *node = malloc(sizeof(struct Node));
    node->key = k;
    node->left = node->right = NULL;
    return node;
}

unsigned int countNodes(struct Node *root) {
    if (root == NULL) return 0;
    return 1 + countNodes(root->left) + countNodes(root->right);
}

bool isCompleteUtil(struct Node *root, unsigned int index, unsigned int numberNodes) {
    if (root == NULL) return true;               /* bos agac complete'tir */
    if (index >= numberNodes) return false;      /* aralikta bosluk var */
    return isCompleteUtil(root->left, 2 * index + 1, numberNodes) &&
           isCompleteUtil(root->right, 2 * index + 2, numberNodes);
}

bool isHeapUtil(struct Node *root) {
    if (root->left == NULL && root->right == NULL) return true;  /* yaprak */
    if (root->right == NULL) return root->key >= root->left->key; /* tek (sol) cocuk */
    if (root->key >= root->left->key && root->key >= root->right->key)
```

```
        return isHeapUtil(root->left) && isHeapUtil(root->right);
    return false;
}

bool isHeap(struct Node *root) {
    unsigned int nodeCount = countNodes(root);
    return isCompleteUtil(root, 0, nodeCount) && isHeapUtil(root);
}

void freeTree(struct Node *r) { if (r) { freeTree(r->left); freeTree(r->right); free(r); } }

void report(struct Node *root) {
    if (isHeap(root)) printf("Given binary tree is a Heap\n");
    else printf("Given binary tree is not a Heap\n");
}

int main(void) {
    /* 10, 9, 8, 7, 6, 5, 4, 3, 2, 1 seviye sirasiyla */
    struct Node *root = newNode(10);
    root->left = newNode(9);
    root->right = newNode(8);
    root->left->left = newNode(7);
    root->left->right = newNode(6);
    root->right->left = newNode(5);
    root->right->right = newNode(4);
    root->left->left->left = newNode(3);
    root->left->left->right = newNode(2);
    root->left->right->left = newNode(1);
    report(root);                       /* heap */

    root->key = 0;                      /* kok 0 yapilirsa */
    report(root);                       /* heap degil */
    root->key = 10;

    struct Node *t = root->left->left->right;   /* aradaki 2 silinir */
    root->left->left->right = NULL;
    free(t);
    report(root);                       /* complete degil -> heap degil */

    freeTree(root);
    return 0;
}
```

Çıktı:

```
Given binary tree is a Heap
Given binary tree is not a Heap
Given binary tree is not a Heap
```

Üçüncü denemede düğüm sayısı 9 olur, ama 1 değerli düğüm hâlâ 9 numaralı indekstedir. $9 \geq 9$ olduğu için `isCompleteUtil` `false` döner.

7. Alıştırma 2: Sırasız Dizide k'ıncı En Küçük Eleman

İkinci soru şuydu: Sırasız bir dizide (unsorted array) k'ıncı en küçük (ya da en büyük) elemanı bulmak isteseydiniz nasıl hareket ederdiniz?

7.1 Üç Adımlı Çözüm

Hoca çözümü üç temel adımla verdi:

1. Bütün elemanlar bir min heap'e yerleştirilir.

2. `extractMin` adlı fonksiyon k kez çağrılır.
3. k'ıncı çağrıda dönen değer, aranan k'ıncı en küçük elemandır.

Bu yolla karmaşıklık $O(n + k \log n)$ seviyesine düşer.

> **Hocanın vurgusu:** Bu önemli bir adım.

7.2 Örnek: 12, 3, 5, 7, 19 Dizisinde 3. En Küçük Eleman

Verilen dizi $12, 3, 5, 7, 19$ ve $k = 3$'tür. Aranan değer 7'dir. Hoca adımları bir öğrenciye anlattırdı ve tahtaya yazdı.

Heap'i kurma (elemanlar tek tek eklenir):

Eklenen	Min heap (dizi)	Açıklama
12	12	İlk eleman köke konur
3	3, 12	3 daha küçük, köke çıkar
5	3, 12, 5	5, 3'ten küçük değil, yerinde kalır
7	3, 7, 5, 12	7, ebeveyni 12'den küçük, yer değiştirir
19	3, 7, 5, 12, 19	19 yaprak olarak kalır, heapify gerekmez

Burada öğrencinin aklına bir soru geldi.

Soru: 7 geldiğinde ağaç zaten complete oluyor; min heap'te yalnız köktekinin en küçük olmasına bakılmıyor mu? **Cevap:** Hayır. Her alt ağaçta ebeveynin değeri kendi çocuklarından küçük olmalıdır. 12 yukarıda, 7 onun çocuğu olarak geldiğinde 7 yukarı, 12 aşağı gitmelidir.

`extractMin` çağrıları:

Çağrı	Dönen	İşlem	Kalan min heap
1	3	Son eleman 19 köke gelir; çocukları 7 ve 5, 5 daha küçük, 19 ile 5 yer değiştirir	5, 7, 19, 12
2	5	Son eleman 12 köke gelir; çocukları 7 ve 19, 7 daha küçük, 12 ile 7 yer değiştirir	7, 12, 19
3	7	Üçüncü çağrının döndürdüğü değer	12, 19

Bir öğrenci, hoca 3'ü kökten alıp son elemanla değiştirdiğinde 3'ün ağaçtan silinmesi gerektiğini hatırlattı. Hoca kökle değiştirdiğini ama silme işlemini henüz yapmadığını söyleyip düzeltti. Son olarak 7 alındığında, sorunun cevabı için ek bir heapify'a gerek kalmaz. Üçüncü çağrıda dönen 7, dizideki üçüncü en küçük elemandır. İz, Python ile doğrulanmıştır.

7.3 Karmaşıklık

Hoca, sıralama yoluna gidilseydi $O(n^2)$'lik bir durumla da karşılaşılabileceğini söyledi. Heap ile k'ıncı en küçük eleman $O(n+k \log n)$ karmaşıklığında bulunur. Elemanları yerleştirme işi n kez yapılır. Heapify ise yalnız k kez, her biri $\log n$ maliyetle yapıldığı için $k \log n$ terimi eklenir.

> **Ders sonrası not:** Heap kurma maliyetinin $O(n)$ olması, bütün dizi üzerinde bir kerede `buildMinHeap` (çocuğu olan son düğümden köke kadar heapify) uygulanırsa geçerlidir. Derste yapıldığı gibi elemanlar tek tek eklenirse kurma en kötü durumda $O(n \log n)$ olur ve toplam $O(n \log n + k \log n)$ olur.

8. Alıştırma 3: Min Heap'i Max Heap'e Dönüştürme

Son soru, bir min heap'i max heap'e dönüştürmekti. Hoca kodun çok kısa olduğunu ve mantığının çok kolay olduğunu söyledi. Kodu paylaşmadan önce bir "beyin jimnastiği" yaptırdı: yeni bir sayfaya $3, 5, 7, 12, 19$ min heap'ini yazdı ve öğrencilerden, son iki derste heap, stack ve queue ile ilgili öğrendiklerini kullanarak bu yapıyı max heap'e dönüştüren algoritmayı tasarlamalarını istedi. Önce 5 dakika düşünme süresi verdi, fikir gelmeyince araya girdi.

8.1 Fikir: Ağacın Kendisi Üzerinde Heapify

Aradan sonra bir öğrenci, ikinci bir ağaç açıp yapraklardan köke doğru özyinelemeyle giderek elemanları yerleştirmeyi önerdi. Hoca bunun bir fikir olduğunu, ama biraz uzun bir yol olduğunu söyledi. Amaç, ekstra bir ağaç kullanmadan dönüşümü ağacın kendisi üzerinde yapmaktır. Yapmak istediğimiz en büyük elemanları en yukarı taşımaktır.

Hoca hangi düğümden başlanacağını sordu. Bir öğrenci "beşinci düğüm" dedi. Hoca düğümleri $0, 1, 2, 3, 4$ diye numaraladı. Kastedilen, değeri 5 olan 1 numaralı düğümdü ve oradan başlamak doğruydu.

8.2 Küçük Örnek: 3, 5, 7, 12, 19

Adım	Düğüm	İşlem	Dizi
Başlangıç		Min heap	3, 5, 7, 12, 19
1	1 (değer 5)	Çocukları 12 ve 19; 19 yukarı çıkar, 5 aşağı iner. 5 yaprağa ulaştı, heapify biter	3, 19, 7, 12, 5
2	0 (değer 3)	Çocukları 19 ve 7; 19 yukarı çıkar, 3 aşağı iner	19, 3, 7, 12, 5
2 (devam)	3 artık 1 numaralı düğümde	Çocukları 12 ve 5; 12 daha büyük, 3 ile 12 yer değiştirir	19, 12, 7, 3, 5

Birinci adımda bir öğrenci heapify'ın özyinelemeyi yukarı mı aşağı mı çağırdığını sordu.

Soru: Heapify'da özyineleme yukarıya doğru mu çağrılıyordu? **Cevap:** Hayır, heapify aşağıya doğru yapılır. 5'in altında başka çocuk olmadığı için burada durur.

İkinci adımda hoca, 3'ün işinin 19 ile yer değiştirince bitmediğini, altında hâlâ çocuklar olduğu için devam ettiğini vurguladı. Bakılacak başka düğüm kalmadığında ağaç max heap özelliğine sahip olur. Min heap olan bir ağaçtan max heap olan bir ağaca dönülmüştür. Sonuç Python ile doğrulanmıştır.

8.3 Büyük Örnek ve İlk Denemedeki Hata

Hoca yanına ikinci, daha büyük bir örnek koydu. Bu da complete bir ikili ağaç ve aynı zamanda bir min heap'ti. Hangi düğümden yola çıkılması gerektiğini sordu. Bir öğrencinin cevabından sonra hoca, 5 numaralı düğümün çocuğu olmadığı için oradan başlanmaması gerektiğini söyledi ve kuralı verdi: $(n-2)/2$'den başlanır. Hoca $n = 9$ alarak $(9-2)/2 = 3{,}5$, yani 3 numaralı düğümden başladı.

Aşağıdaki iz bu diziyle verilmiştir (ara adım eklendi: dizi hâlleri):

Adım	Düğüm	İşlem	Dizi
Başlangıç		Min heap	3, 5, 9, 6, 8, 20, 10, 18, 12, 9

Adım	Düğüm	İşlem	Dizi
1	3 (değer 6)	Çocukları 18 ve 12; 18 yukarı, 6 aşağı. 6 yaprağa ulaştı	3, 5, 9, 18, 8, 20, 10, 6, 12, 9
2	2 (değer 9)	Çocukları 20 ve 10; 20 yukarı, 9 aşağı. 9 yaprağa ulaştı	3, 5, 20, 18, 8, 9, 10, 6, 12, 9
3	1 (değer 5)	18 ile yer değiştirir; 5 yaprağa ulaşmadı, 12 ile yer değiştirir	3, 18, 20, 12, 8, 9, 10, 6, 5, 9
4	0 (değer 3)	20 ile yer değiştirir; 3'ün işi bitmedi, 10 ile yer değiştirir	20, 18, 10, 12, 8, 9, 3, 6, 5, 9

Hoca, oluşan ağacın max heap olup olmadığını sordu. Bir öğrenci 4 numaralı düğüme bakılmadığını söyledi. Gerçekten de 4 numaralı düğümdeki 8'in çocuğu 9'dur ve $8 < 9$ olduğu için sonuç max heap değildir.

Hocanın vurgusu: Demek ki çocuğu olan düğümlere mutlaka bakmamız gerekiyor.

8.4 Doğru Başlangıçla Yeniden Çözüm

Sohbetten $(n-1)/2$ önerisi geldi. Hoca bunun indeksin 0'dan mı 1'den mi başlatıldığıyla ilgili olduğunu söyledi ve $n = 9$ için $(9-1)/2 = 4$ alarak 4 numaralı düğümden yeniden başladı.

Adım	Düğüm	İşlem	Dizi
Başlangıç		Min heap	3, 5, 9, 6, 8, 20, 10, 18, 12, 9
1	4 (değer 8)	Çocuğu 9; 9 yukarı, 8 aşağı	3, 5, 9, 6, 9, 20, 10, 18, 12, 8
2	3 (değer 6)	18 yukarı, 6 aşağı	3, 5, 9, 18, 9, 20, 10, 6, 12, 8
3	2 (değer 9)	20 yukarı, 9 aşağı	3, 5, 20, 18, 9, 9, 10, 6, 12, 8
4	1 (değer 5)	18 ile, sonra 12 ile yer değiştirir	3, 18, 20, 12, 9, 9, 10, 6, 5, 8
5	0 (değer 3)	20 ile, sonra 10 ile yer değiştirir	20, 18, 10, 12, 9, 9, 3, 6, 5, 8

Son dizi max heap'tir (Python ile doğrulandı). Hoca, her seferinde heapify yaparak min heap'in max heap'e dönüştürüldüğünü söyledi.

Hocanın vurgusu: Önemli olan, çocuğu olan (son) düğümden başlamak ve o düğümden geriye doğru köke kadar ilerleyip bütün elemanları yukarıdan aşağı (top-to-bottom) heapify işleminden geçirerek ağacın yeniden şekillenmesini sağlamaktır.

Ders sonrası not: Hocanın verdiği $\lfloor (n-2)/2 \rfloor$ kuralı 0 tabanlı dizi için doğrudur ve $\lfloor n/2 \rfloor - 1$ ile aynı sonucu verir. İlk denemedeki hata kuralda değil, eleman sayısındadır: ağaçta 4 numaralı düğümün bir çocuğu olduğuna göre en az 10 düğüm vardır ($n = 10$) ve $\lfloor (10-2)/2 \rfloor = 4$'tür. Sohbette önerilen $\lfloor (n-1)/2 \rfloor$ de güvenlidir; bazı n değerlerinde fazladan bir yaprakta etkisiz bir heapify çağrısı yapar. 1 tabanlı dizide çocuğu olan son düğüm $\lfloor n/2 \rfloor$'dir.

Ek bilgi (derste anlatılmadı): Min heap'i max heap'e dönüştürmek için min heap olmasının bir önemi yoktur. Yapılan işlem, herhangi bir diziyi max heap'e çeviren `buildMaxHeap` işleminin aynısıdır. Bu işlem toplamda $O(n)$ sürede çalışır, çünkü düğümlerin çoğu alt seviyelerdedir ve onların heapify maliyeti küçüktür.

Örnek Sorular

Bu derse ait hazır kare ve video yoktur. Aşağıdaki sorular derste sözlü sorulan ve çözülen sorulardır.

Soru 1 (derste sözlü, 00:16:42): Öncelikli kuyruğu gerçeklemek için heap dışında hangi veri yapılarını kullanabiliriz?

Çözüm (derste verildi): Dizi, bağlı liste ve dengeli ikili arama ağacı (AVL vb.). Yığın, istenen verimlilikte bir öncelikli kuyruk sağlamaz. Dengesiz bir ağacın faydası yoktur.

Soru 2 (derste sözlü, 00:20:47 ve 00:22:52): Öncelikli kuyruk dizi, sıralı bağlı liste ve AVL ağacı ile gerçeklenirse `insert`, `remove` ve `peek` işlemlerinin karmaşıklıkları nedir?

Çözüm (derste verildi): Dizi için $O(1)$, $O(n)$, $O(n)$. Sıralı bağlı liste için $O(n)$, $O(1)$, $O(1)$. AVL için `insert` ve `remove` $O(\log n)$. Ayrıntı 2.1-2.4'te.

Soru 3 (derste sözlü, 00:26:26): İkili arama ağacında `peek` işlemi $O(1)$ midir, $O(\log n)$ midir? `delete` neden $O(\log n)$'dir?

Çözüm (derste verildi, 00:52:32): Hoca $O(1)$ dedi; en büyük değer en sağdaki düğümdedir. `delete`'te ise silmeden sonraki yeniden düzenleme $O(\log n)$'dir. En sağdaki düğüme kökten inmek $O(\log n)$ sürdüğü için $O(1)$ ancak ayrı bir işaretçiyle mümkündür (bkz. 2.5'teki ders sonrası not).

Soru 4 (derste sözlü, 00:36:27): Slayttaki `insert` kodu ile tahtada yapılan ekleme arasındaki fark nedir?

Çözüm (derste verildi, 00:54:13): Slayttaki kod eklemeden sonra $size/2 - 1$'den köke kadar bütün iç düğümleri heapify eder. Ağaç zaten heap olduğu için bu gereksizdir; yalnız yeni elemanın aşağıdan yukarı yerleşmesi yeterlidir.

Soru 5 (derste sözlü, 00:56:12): Verilen bir ikili ağacın heap özelliği taşıyıp taşımadığını bulan algoritmayı yazın.

Çözüm (derste verildi): Önce düğümler sayılır ve ağacın complete olduğu indeks yöntemiyle kontrol edilir, sonra her ebeveynin çocuklarından büyük olduğu özyinelemeyle kontrol edilir. Kod 6.5'te.

Soru 6 (derste sözlü, 01:07:38 ve 01:12:01): Sırasız bir dizide k'ıncı en küçük elemanı nasıl bulursunuz? $12, 3, 5, 7, 19$ dizisinde 3. en küçük elemanı adım adım bulun.

Çözüm (derste verildi): Elemanlar min heap'e yerleştirilir, `extractMin` 3 kez çağrılır; dönen değerler 3, 5, 7'dir ve cevap 7'dir. Karmaşıklık $O(n + k \log n)$. İz 7.2'de.

Soru 7 (derste sözlü, 01:18:15): $3, 5, 7, 12, 19$ min heap'ini max heap'e dönüştüren algoritmayı tasarlayın.

Çözüm (derste verildi, 01:37:01): Çocuğu olan son düğümden (1 numaralı düğüm) köke kadar her düğüm için yukarıdan aşağı max heapify uygulanır. Sonuç $19, 12, 7, 3, 5$'tir. İz 8.2'de.

Soru 8 (derste sözlü, 01:40:21 ve 01:43:22): İkinci, daha büyük min heap örneğinde dönüşüme hangi düğümden başlanmalıdır? 3 numaralı düğümden başlanınca sonuç neden max heap olmadı?

Çözüm (derste verildi): 4 numaralı düğümün bir çocuğu vardı ve atlandı; çocuğu olan her düğüme bakılmalıdır. Doğru başlangıç 4 numaralı düğümdür. İz 8.3 ve 8.4'te.

Dersin Toparlaması

Bu derste öncelikli kuyruğun yalnız heap ile değil, dizi, bağlı liste ve dengeli ikili arama ağacıyla da gerçeklenebileceği görüldü. Her gerçeklemede `insert`, `remove` ve `peek` işlemlerinin maliyeti farklıdır. Binary heap, ekleme ve çıkarmayı $O(\log n)$'e indirip en öncelikli elemana $O(1)$'de eriştirdiği için genelde tercih edilir. Banka örneğinde yeni gelen daha öncelikli müşterinin aşağıdan yukarı heapify ile öne geçtiği, hizmet alınca kökün son elemanla değiştirilip yukarıdan aşağı heapify yapıldığı gösterildi. Eklemede bütün iç düğümleri gezmenin gereksiz olduğu, o döngünün yalnız sıfırdan heap kurarken ve heap sort'ta kullanıldığı vurgulandı.

Geçen haftanın üç alıştırması çözüldü: bir ikili ağacın heap olup olmadığını complete ve heap sırası kontrolleriyle bulmak, k'ıncı en küçük elemanı min heap ve k kez `extractMin` ile $O(n + k \log n)$'de bulmak ve bir min heap'i, çocuğu olan son düğümden köke kadar max heapify uygulayarak max heap'e dönüştürmek. Hoca, öncelikli kuyruktan sonra graflara ve minimum yayılan ağaç (minimum spanning tree) için Prim ve Kruskal algoritmalarına geçileceğini, 13. haftada metin (string) arama algoritmalarının, son haftada da arama algoritmalarının karşılaştırmasının ele alınacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.